

LAB3_P2

May 4, 2026

AI TECH - Akademia Innowacyjnych Zastosowań Technologii Cyfrowych. Programu Operacyjnego Polska Cyfrowa na lata 2014-2020 AI-TECH

Przykład 2 DO LABORATORIUM 3

z UCZENIA MASZYNOWEGO

Klasyfikacja nienadzorowana

Adam Kurowski

Dobór liczby klas w algorytmie k średnich i klasteryzacji hierarchicznej

</html>



Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego Program Operacyjny Polska Cyfrowa na lata 2014-2020, Oś Priorytetowa nr 3 “Cyfrowe kompetencje społeczeństwa” Działanie nr 3.2 “Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej” Tytuł projektu: „Akademia Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)”

1 Przykład 2.: Dobór liczby klastrów w algorytmie k średnich i klasteryzacji hierarchicznej

1.1 Analiza struktury danych wejściowych

W tym przykładzie zajmiemy się już zagadnieniem nienadzorowanej klasyfikacji wysokowymiarowych zbiorów danych. Procesu tego już nie będzie dało się przedstawić w prosty sposób za pomocą obszarów decyzyjnych na dwuwymiarowej płaszczyźnie. Nadal jednak możliwe będzie zapoznanie się z efektami działania wybranych algorytmów (k średnich i hierarchicznego) za pomocą wizualizacji już bez pokazania granic obszarów przypisanych do poszczególnych klastrów. Granice te teraz będą już odnajdowane w przestrzeni wysokowymiarowej.

```
[2]: # Import bibliotek
import numpy as np # obliczenia numeryczne
```

```

import pandas as pd # przetwarzanie danych "tabelarycznych"
import matplotlib.pyplot as plt # wizualizacja danych
from sklearn.datasets import load_diabetes # zbiory danych
from sklearn.decomposition import PCA # redukcja wymiarowości
from sklearn.cluster import KMeans, AgglomerativeClustering # implementacja
    ↪ algorytmu k średnich
                                                    # i klasteryzacji
    ↪ hierarchicznej
import scipy.cluster.hierarchy as sch # procedury potrzebne do wizualizacji
    ↪ działania algorytmów hierarchicznych
import copy as cp # do wykonywania kopii struktur w przypadku, gdy nie chcemy
    ↪ zmodyfikować oryginalnych danych

```

Do dyspozycji w trakcie ćwiczenia będziemy posiadać zbiory danych [Boston Housing](#) i [Diabetes](#), które także możemy łatwo pozyskać go za pomocą API udostępnianego przez bibliotekę scikit-learn, w której [dokumentacji](#) przedstawione zostały także dodatkowe informacje na ich temat. Tym razem nie będą to już zbiory przewidziane do ilustracji zadań klasyfikacji, tylko zbiory przeznaczone do testowania algorytmów regresji. Może się to wydawać nieintuicyjne, bo przecież testować będziemy algorytmy klasyfikacji nienadzorowanej, jednak taki dobór danych pozwala na pokazanie jednego z praktycznych zastosowań klasyfikacji nienadzorowanej. W przykładzie zademonstrowany zostanie przykład poszukiwania klas danych, które wyróżniają się pod względem metryki typowo estymowanej w zadaniach regresji. Dla każdego ze zbiorów poszukamy klasy elementów (mieszkań, lub zestawów danych opisujących pacjentów) wskazujących na pewne szczególne klasy:

w zbiorze Boston Housing poszukamy klasy mieszkań, które są szczególnie wartościowe na tle innych nieruchomości, o których dane znajdują się w zbiorze,

w zbiorze Diabetes poszukamy osób, których dane medyczne wskazują na to, że są zagrożone szczególnie szybkimi postępami choroby.

W ramach obowiązkowej części ćwiczenia skupimy się na zbiorze Boston Housing. Chętni studenci są jednak zachęceni do wypróbowania nowo pozyskanej wiedzy na dodatkowym zbiorze, jakim jest Diabetes, jeżeli uda im się szybko wykonać pozostałe zadania laboratoryjne.

```

[3]: # Załadowanie wybranego zbioru danych
# dataset_dict = load_boston() zostało usunięte z nowszych wersji sklearn,
    ↪ pobieramy ręcznie
data_url = "http://lib.stat.cmu.edu/datasets/boston"
raw_df = pd.read_csv(data_url, sep="\s+", skiprows=22, header=None)
data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
target = raw_df.values[1::2, 2]
feature_names = np.array(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE',
    ↪ 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT'])

dataset_dict = {
    'data': data,
    'target': target,
    'feature_names': feature_names
}

```

```
# Wypisanie podstawowych informacji o danych pozyskanych poprzez API biblioteki ↵
↳ scikit-learn.
print('Słownik przechowujący zbiór danych ma następujące klucze:')
print(list(dataset_dict.keys()))

print()
print('Do dyspozycji są następujące cechy:')
print(dataset_dict['feature_names'])

# Kilka kluczy ze słownika dla wygody wydzielimy do zmiennych:
data = dataset_dict['data']
target = dataset_dict['target']
```

Słownik przechowujący zbiór danych ma następujące klucze:
['data', 'target', 'feature_names']

Do dyspozycji są następujące cechy:
['CRIM' 'ZN' 'INDUS' 'CHAS' 'NOX' 'RM' 'AGE' 'DIS' 'RAD' 'TAX' 'PTRATIO'
'B' 'LSTAT']

W tym przykładzie także będziemy badać wpływ normalizacji. Odpowiedni kod jest zamieszczony w komórce poniżej. Na początku nie będziemy stosować normalizacji. Jej wpływ będzie badany w dalszej części ćwiczenia.

```
[4]: # normalizacja do zakresu od -1 do 1
def norm_unit_absval(data):
    data_norm = cp.copy(data)
    for i in range(data_norm.shape[1]):
        data_norm[:,i] = data_norm[:,i]-np.min(data_norm[:,i])
        data_norm[:,i] = data_norm[:,i]/np.max(data_norm[:,i])
        data_norm[:,i] = data_norm[:,i]*2 - 1
    return data_norm

# standaryzacja
def norm_standardize(data):
    data_norm = cp.copy(data)
    for i in range(data_norm.shape[1]):
        data_norm[:,i] = data_norm[:,i]-np.mean(data_norm[:,i])
        data_norm[:,i] = data_norm[:,i]/np.std(data_norm[:,i])
    return data_norm

# Odkomentować, jeżeli normalizacja jest potrzebna:
data_norm = cp.copy(data) # Odkomentować jeśli nie korzystamy z normalizacji
# data_norm = norm_unit_absval(data)
# data_norm = norm_standardize(data)
```

W przykładzie 2. do wizualizacji posłużymy się tylko algorytmami redukcji wymiarowości za pomocą PCA. Punkty uzyskane w ten sposób zostaną oznaczone kolorem w zależności od wartości

metryki stanowiącej daną wejściową lub daną przybliżaną, która w przypadku pokazywanym w tym przykładzie będzie służyć do wyłonienia wyróżniających się klas danych. Ich współrzędne policzone są w komórce poniżej:

```
[5]: # Obiekt algorytmu redukcji wymiarowości do 2 wymiarów.
pca = PCA(n_components=2)

# Efekt redukcji wymiarowości zapiszemy w osobnej zmiennej:
reduced_data = pca.fit_transform(data_norm)
```

Wizualizacja przetwarzanego zbioru danych może być zrealizowana za pomocą kodu podanego w komórce poniżej. Warto też zapoznać się z rozkładem wartości przechowywanych w poszczególnych wymiarach zbioru danych. Rozkład ten zwizualizowany jest za pomocą mapy koloru.

```
[6]: # W zmiennej coloring_variable_name można wybrać zmienną, która wykorzystana
    ↪będzie do kolorowania
# punktów na zobrazowaniu wykorzystującym PCA, jest to przydatne by zorientować
    ↪się, jaki wpływ na umiejscowienie
# punktów mają poszczególne wymiary zbioru danych. Poniżej podane są nazwy
    ↪poszczególnych wymiarów:
#
# - w zbiorze Boston Housing do dyspozycji są następujące wymiary:
# 'CRIM' 'ZN' 'INDUS' 'CHAS' 'NOX' 'RM' 'AGE' 'DIS' 'RAD' 'TAX' 'PTRATIO' 'B'
    ↪'LSTAT'
#
# - w zbiorze Diabetes do dyspozycji są z kolei zmienne:
# 'age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6'

# Jeśli wybrana ma być zmienna stanowiąca pierwotnie cel regresji w zbiorach
    ↪danych, to należy zmiennej coloring_variable_name
# nadać wartość 'target'.
#
# W zbiorze Boston Housing będzie to oznaczać medianę ceny nieruchomości w
    ↪obszarze,
# którego dotyczy punkt, wyrażoną w tysiącach dolarów.
#
# W zbiorze Diabetes, zmienna target oznaczać będzie ilościową metrykę
    ↪informującą o postępie choroby - im większa
# jej wartość, tym większy postęp choroby w rok po dokonaniu pomiaru
    ↪pozostałych wartości w zbiorze danych.

coloring_variable_name = 'target'

# Sprawdzamy, czy mamy wybrać zmienną target, czy kolumnę w tablicy data_norm.
if coloring_variable_name == 'target':
    coloring_variable = target
else:
```

```

dim_index      = np.where(dataset_dict['feature_names'] == '
↳coloring_variable_name)
coloring_variable = data_norm[:,dim_index].squeeze()

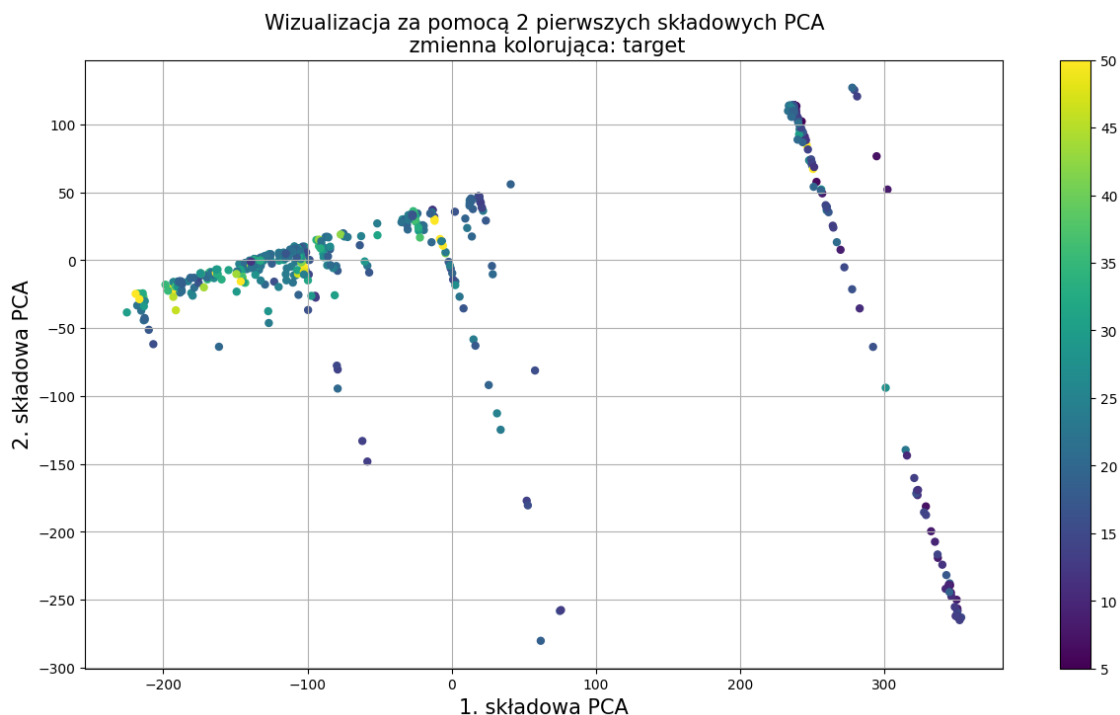
# Zdefiniowanie rozmiarów wykresu
plt.figure(figsize=(15,8))

# Opisujemy osie
plt.xlabel('1. składowa PCA', fontsize=15)
plt.ylabel('2. składowa PCA', fontsize=15)
plt.scatter(reduced_data[:,0],reduced_data[:,1], c=coloring_variable, s=25)

# Na koniec dodajemy tytuł, legendę i rysujemy siatkę.
plt.colorbar()
plt.title(f'Wizualizacja za pomocą 2 pierwszych składowych PCA\n
↳zmienna kolorująca: {coloring_variable_name}', fontsize=15)
plt.grid()

# W razie potrzeby można pomniejszyć lub zmienić wizualizowany fragment za
↳pomocą funkcji xlim i ylim.
# plt.xlim([-2.5,2.5])
# plt.ylim([-2.5,2.5])

```



Separacja międzyklasowa w zbiorze wine ma tę cechę, że zawsze znajdziemy klaster, który blisko

sąsiaduje z innym klastrem. Zobaczmy, jak poradzą sobie z tym faktem algorytmy klasyfikacji nienadzorowanej.

1.2 Algorytm k średnich

Do znalezienia grup punktów danych, wyróżniających się na tle wybranej metryki docelowej posłużymy się najpierw algorytmem k średnich. Tu jednak pojawia się pierwszy problem. Konieczna jest decyzja odnośnie dokładnej wartości parametru k . Z pomocą może przyjść tu prosta metoda polegająca na obliczaniu średniego dystansu pomiędzy punktami w klastrach generowanych przez algorytm k średnich. Jest to metoda poszukiwania punktu przegięcia funkcji, która wiąże liczbę klastrów k z średnim dystancem pomiędzy punktami w klastrze (ang. *elbow method*). Wymaga ona obliczenia algorytmu k średnich wiele razy i wykreślenia odpowiedniego wykresu. Na szczęście biblioteka scikit-learn umożliwia pobranie metryki będącej sumą podniesionych do kwadratu dystansów między punktami. Jest to parametr o angielskiej nazwie *inertia*, który dalej dla wygody będziemy dalej nazywać inercją.

Punkt przegięcia wykresu tej wartości pomoże w wyborze odpowiedniej liczby klastrów. Funkcja ta jest zawsze malejąca bo wraz ze wzrostem liczby klastrów dystanse między punktami zawsze maleją. Punkt przegięcia jednak wyznacza tę liczbę klastrów, dla której ten spadek przestaje być gwałtowny. Zakładamy, że dzieje się tak dlatego, że właśnie w tym punkcie znaleźliśmy wartość k najlepiej dopasowaną do wewnętrznej struktury analizowanych przez nas danych.

```
[8]: # Listy w których będziemy zapisywać poszczególne wartości parametru k i
    ↪ odpowiadających im wartości
    # metryki uśrednionej odległości pomiędzy punktami.
    k_values = []
    inertia_scores_vec = []

    # Następnie w pętli wywołujemy algorytm k średnich i zapisujemy wartości k oraz
    ↪ intercji.
    # Tym razem wykorzystamy domyślne nastawy pracy algorytmu przyjęte w bibliotece
    ↪ scikit-learn.
    for k in range(2,15):
        clusterer = KMeans(n_clusters=k)
        clusterer.fit(data_norm)
        inertia_scores_vec.append(clusterer.inertia_)
        k_values.append(k)

    # Na koniec rysujemy wykres inercji w funkcji liczby klastrów k.
    fig = plt.figure(figsize=(15,8))
    plt.plot(k_values,inertia_scores_vec, marker='o', c='darkblue')
    plt.grid()
    # Na dodatkowej (prawej) osi y narysujemy drugą pochodną funkcji inercji, co
    ↪ pozwoli łatwiej odnaleźć punkt
    # gdzie dynamika zmian inercji (przyspieszenie) jest możliwie mała. W
    ↪ najlepszym przypadku - będzie to minimum
    # modułu drugiej pochodnej.
    ax1 = plt.gca()
```

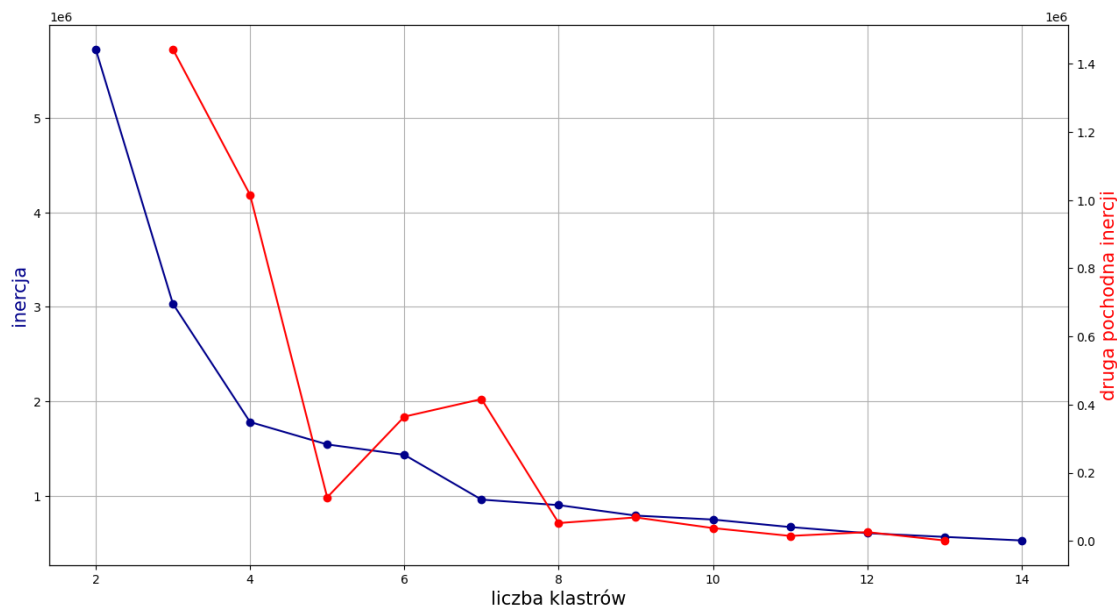
```

ax2 = ax1.twinx()
module_of_second_derivative = np.abs(np.diff(np.diff(inertia_scores_vec)))
ax2.plot(k_values[1:-1], module_of_second_derivative, marker='o',color='red')

# Opis osi
ax1.set_xlabel('liczba klastrów', fontsize=15)
ax1.set_ylabel('inercja', color='darkblue', fontsize=15)
ax2.set_ylabel('druga pochodna inercji',color='red', fontsize=15)

```

[8]: `Text(0, 0.5, 'druga pochodna inercji')`



Przy domyślnych ustawieniach analizy w przykładzie - pierwsze minimum modułu drugiej pochodnej inercji występuje dla wartości $k = 5$, co sugeruje, że to właśnie dla tej liczby klastrów warto wykonać pierwszą analizę. Należy pamiętać, że niestety w ogólności nie istnieje gwarancja, że tego typu minimum będzie istnieć. Stąd dla bardzo trudnych przypadków dobór liczby klastrów nawet na podstawie wykresu inercji jest uznaniowy.

```

[9]: # Liczba klastrów, której się spodziewamy na podstawie wykresu inercji.
n_clusters = 5

# Wykonanie algorytmu k średnich
clusterer = KMeans(n_clusters=n_clusters)
clusterer.fit(data_norm)
k_means_predictions = clusterer.predict(data_norm)

# Wizualizacja poprzez oznaczenie klastrów kolorami na efekcie algorytmu PCA.
plt.figure(figsize=(15,8))

```

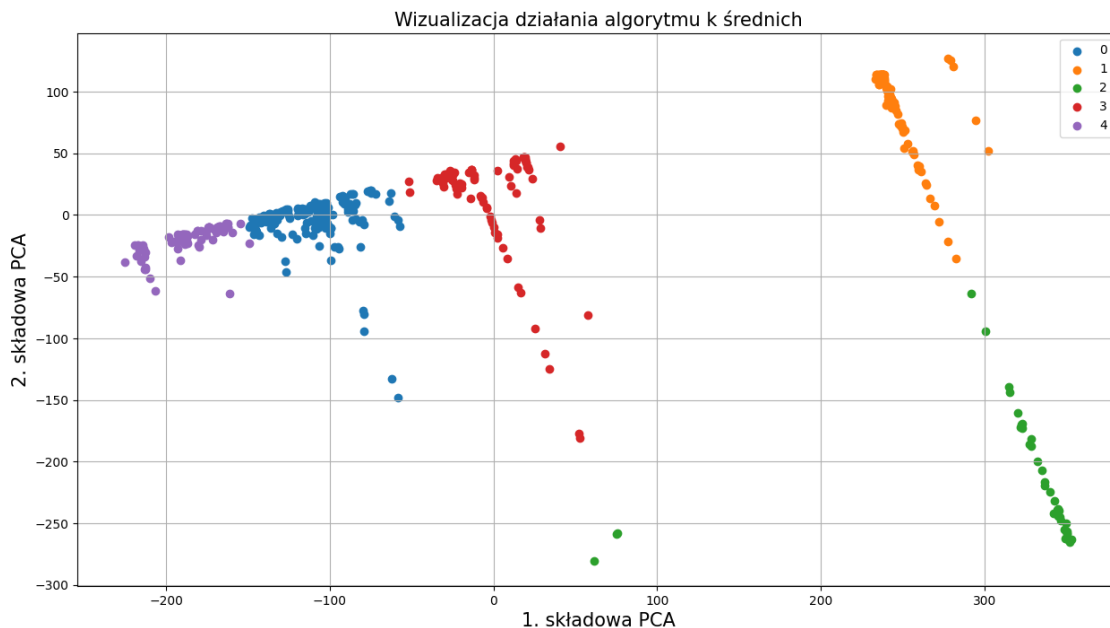
```

# Narysujemy wykres punktowy z rozdzieleniem danych na klasy.
for val in np.unique(k_means_predictions):
    # Generujemy sobie maskę za pomocą której wybierzemy punkty należące do
    ↪każdej z klas.
    mask = k_means_predictions==val
    # Dla tak wszystkich punktów wybranych za pomocą odpowiadającej danej
    ↪klasie maski rysujemy wykres punktowy.
    plt.scatter(reduced_data[mask,0],reduced_data[mask,1], label=val)

# Oznaczenie osi i dodatkowe elementy wykresu.
plt.xlabel('1. składowa PCA', fontsize=15)
plt.ylabel('2. składowa PCA', fontsize=15)
plt.title("Wizualizacja działania algorytmu k średnich", fontsize=15)
plt.grid()
plt.legend(fontsize=10)

```

[9]: <matplotlib.legend.Legend at 0x1fb6a87f8c0>



Zadaniem w ramach przykładu jest odszukanie takich skupisk punktów danych, które istotnie odróżniają się od innych skupisk, jeżeli porównać powiązane z nimi wartości metryki docelowej. Aby móc sprawdzić te wartości posłużymy się wykresem pudełkowym. Pozwala on na łatwie graficzne zwizualizowanie podstawowych statystyk dotyczących każdego klastra, dokładniejszy opis objaśniający w jaki sposób należy interpretować wykres pudełkowy można odnaleźć na przykład w [dokumentacji biblioteki Pandas](#).


```

[10]: # W wizualizacji cech klastrow znowu będzie możliwy wybór wizualizowanej
      ↪ wielkości - zmienna docelowa to wartość 'target',
      # wymiar pochodzący ze zbioru danych natomiast, to odpowiednia nazwa wymiaru:
      #
      # - w zbiorze Boston Housing do dyspozycji są następujące wymiary:
      # 'CRIM' 'ZN' 'INDUS' 'CHAS' 'NOX' 'RM' 'AGE' 'DIS' 'RAD' 'TAX' 'PTRATIO' 'B'
      ↪ 'LSTAT'
      #
      # - w zbiorze Diabetes do dyspozycji są z kolei zmienne:
      # 'age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6'

      # Nazwa wizualizowanej wielkości:
      visualized_quantity_name = 'target'

      # Sprawdzamy, czy mamy wybrać zmienną target, czy kolumnę w tablicy data_norm.
      if visualized_quantity_name == 'target':
          visualized_quantity = target
      else:
          dim_index = np.where(dataset_dict['feature_names'] ==
          ↪ visualized_quantity_name)
          visualized_quantity = data_norm[:,dim_index].squeeze()

      # Proces wizualizacji aczniemy od pustej listy.
      clusterized_income_df = []

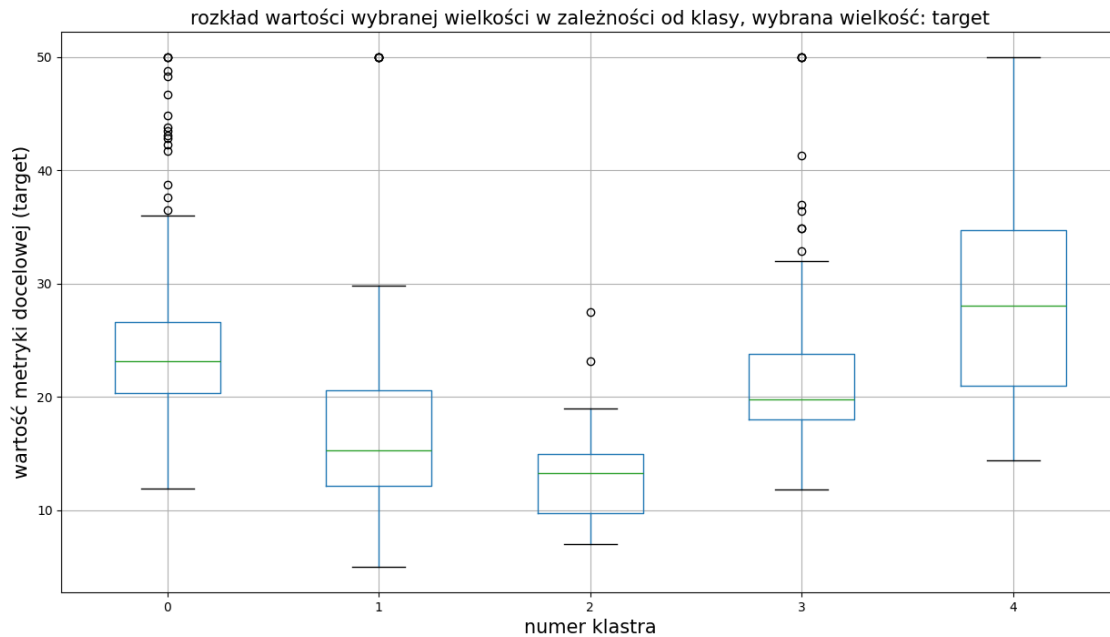
      # W pętli dodajemy do listy wartości metryki docelowej (target), z której za
      ↪ pomocą maski
      # wybieramy wartości powiązane z każdym z klastrow.
      for val in np.unique(k_means_predictions):
          mask = k_means_predictions==val
          clusterized_income_df.append(visualized_quantity[mask])

      # Na koniec konwertujemy listę do ramki danych (DataFrame)
      clusterized_income_df = pd.DataFrame(clusterized_income_df).T

      # Na podstawie tak przygotowanej ramki danych można w prosty sposób przygotować
      ↪ wykres pudełkowy za pomocą
      # metody .boxplot().
      plt.figure(figsize=(15,8))
      clusterized_income_df.boxplot()
      plt.xlabel('numer klastra', fontsize=15)
      plt.ylabel('wartość metryki docelowej (target)', fontsize=15)
      plt.title(f'rozkład wartości wybranej wielkości w zależności od klasy, wybrana
      ↪ wielkość: {visualized_quantity_name}',
                fontsize=15)

```

```
[10]: Text(0.5, 1.0, 'rozkład wartości wybranej wielkości w zależności od klasy,  
wybrana wielkość: target')
```



Przy domyślnych ustawieniach analizy w przykładzie - jesteśmy w stanie zaobserwować, że udało się odszukać skupisko (klasę, klastę) nieruchomości szczególnie wartościowych, jak i klasy, które powiązane są z tymi o najmniejszych wartościach (np. klasa nr 3).

1.3 Algorytm hierarchiczny

Algorytm k średnich nie jest jedynym możliwym sposobem nienadzorowanej klasyfikacji danych. Tzw. podejście hierarchiczne jest innym przykładem metody, w której punkty łączone są w skupienia w sposób niewymagający definiowania klas danych. Łączenie to odbywa się poprzez dobór takich punktów (a później - skupisk punktów), których połączenie w większe grupy powoduje najmniejszy możliwy wzrost metryki opisującej sposób podziału danych na klasy. W bibliotece scikit-learn algorytm taki zaimplementowany jest w postaci obiektu [AgglomerativeClustering](#). Łączenie punktów w skupiska może opierać się na minimalizacji przyrostu czterech możliwych metryk:

metryki Warda, której użycie minimalizuje wariancję tworzonych klast,ów,

średniego dystansu pomiędzy punktami znajdującymi się w różnych klast,ach,

maksymalnej odległości pomiędzy dwoma punktami znajdującymi się w różnych klast,ach,

minimalnej odległości pomiędzy dwoma punktami znajdującymi się w różnych klast,ach.

Przebieg procesu łączenia pojedynczych punktów w klast,ry można prześledzić za pomocą tzw *dendrogramu*. Jest to rodzaj wizualizacji, który pokazuje jak zachowuje się wartość wybranej metryki wraz z łączeniem ze sobą kolejnych skupisk danych. Na dole dendrogramu znajdują się pojedyncze

punkty danych, na jego górze znajduje się część odpowiadająca wartości metryki dla podziału danych na dwa skupiska (klasy).

Dendrogram można wykorzystać do określenia liczby klas na które można podzielić klasyfikowane w sposób nienadzorowany dane. Zależy nam na wybraniu takiej liczby klastrow, dla której podział na mniejszą lub większą liczbę klastrow powoduje znaczną, największą możliwą zmianę metryki. Podział taki charakteryzowany jest przez ten obszar dendrogramu, dla którego widoczne są tylko pionowe kreski oznaczające poszczególne klastry (kreski poziome oznaczają podziału). Jest to tożsame z powiedzeniem, że jest to taka liczba klastrow, dla których odległość pomiędzy poziomymi markerami podziałów jest największa. Przykładowy dendrogram dla danych przetwarzanych w tym przykładzie jest widoczny poniżej:

```
[11]: # Dostępne metryki łączenia klastrow:
#      'ward'      - metryka minimalizująca wariancję łączonych klastrow
#      'average'   - maksymalizuje średni dystans pomiędzy punktami
#                   w różnych klastrach
#      'complete'  - minimalizuje maksymalną odległość
#                   między punktami w klastrach
#      'single'    - maksymalizuje najmniejszą odległość
#                   między punktami w klastrach

# Wybieramy metrykę według której łączymy poszczególne punkty danych
linkage_method = 'average'

# Rysujemy dendrogram
plt.figure(figsize=(15,8))
dend = sch.dendrogram(sch.linkage(data_norm, method=linkage_method),
    ↳truncate_mode='lastp', p=60)
plt.ylabel(f'metryka łączenia klastrow ({linkage_method})', fontsize=15)
plt.title(f'dendrogram algorytmu hierarchicznego dla metryki:
    ↳{linkage_method}', fontsize=15)

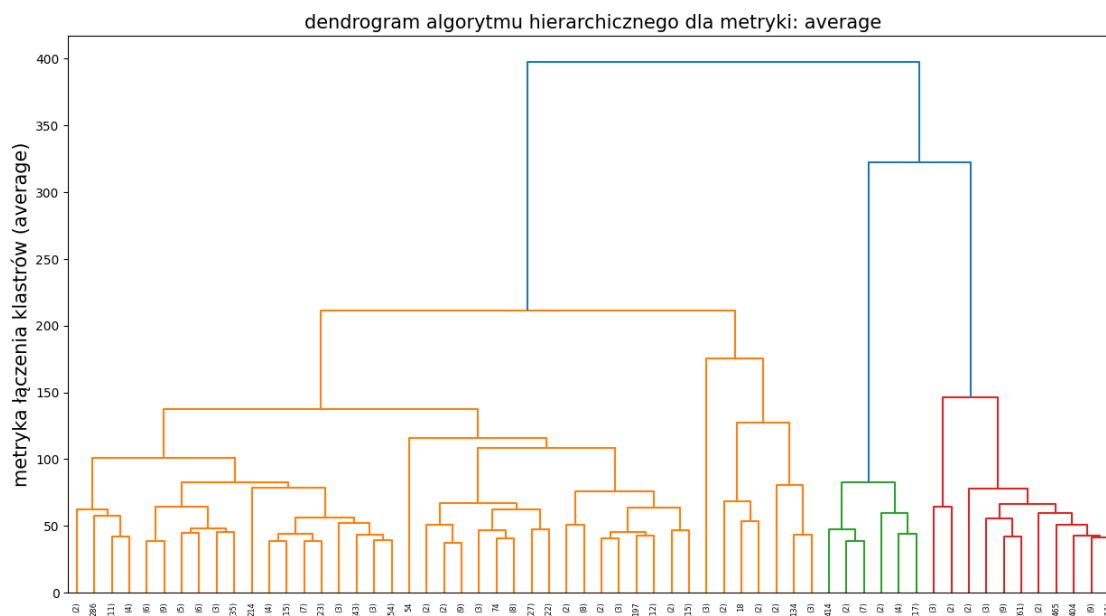
# Odczytywanie informacji o liczbie klastrow może nie być na początku
    ↳intuicyjne. Dendrogram to schemat pokazujący wzrost
# wybranej metryki (u nas podawanej w zmiennej linkage_method) wraz z łączeniem
    ↳mniejszych grup punktów w coraz większe
# klastry. Punkty łączenia symbolizowane są przez poziome kreski. Naszym
    ↳zadaniem jest znaleźć takie łączenia
# (poziome kreski), pomiędzy którymi jest największy odstęp - oznacza to, że
    ↳jest to najbardziej "kosztowne" połączenie
# w historii działań wykonywanych przez algorytm.
# Pionowe kreski odpowiadają poszczególnym klastrom istniejącym w danej chwili,
    ↳dlatego po wyznaczeniu tego
# najbardziej kosztownego łączenia wystarczy policzyć, ile klastrow
    ↳symbolizowanych przez pionowe kreski)
# jest obecnych w stanie przed tym najbardziej kosztownym połączeniem
#
```

```
# Przykładowo, dla domyślnych nastaw skryptów, możemy taki obszar odłączenia
↳ najbardziej kosztownego (wyżej
# umiejscowionego na dendrogramie) do jego poprzednika (położonego niżej na
↳ dendrogramie)..

# (domyślnie linijki są zakomentowane, by pokazać jak wygląda dendrogram,
↳ odkomentuj aby zobaczyć sposób doboru
# wartości k na podstawie dendrogramu)
# plt.gca().axhline(211, c='k') # "dolny" sąsiad najbardziej kosztownego
↳ połączenia
# plt.gca().axhline(323, c='k') # najbardziej kosztowne połączenie klastrow
# plt.gca().axhline(260, c='k', linestyle='--') # najbardziej kosztowne
↳ połączenie klastrow
# plt.gca().text(10, 270, 'linia przerywana przecina 3 pionowe kreski,
↳ optymalne k to 3') # najbardziej kosztowne połączenie klastrow

# pomiędzy łączeniami mamy 3 kreski oznaczające, że przed wykonaniem
↳ najbardziej kosztownego (niekorzystnego) łączenia
# istnieją 3 klastry i taka właśnie jest optymalna liczba klastrow odczytana z
↳ dendrogramu.
```

```
[11]: Text(0.5, 1.0, 'dendrogram algorytmu hierarchicznego dla metryki: average')
```



Na powyższym dendrogramie, dla domyślnych danych, widoczny jest szczególnie długi obszar bez podziałów klastrow występujący dla 3 klastrow (długość mierzymy od najwyższej pomarańczowej poziomej linii do najniższej poziomej linii niebieskiej). Stąd w kodzie poniżej dla domyślnych danych, dla metryki powiązanej ze średnim dystansem pomiędzy punktami przyjmujemy, że $k = 3$.

```
[12]: # Liczba klastrow, której się spodziewamy na podstawie wyglądu dendrogramu.
n_clusters = 4

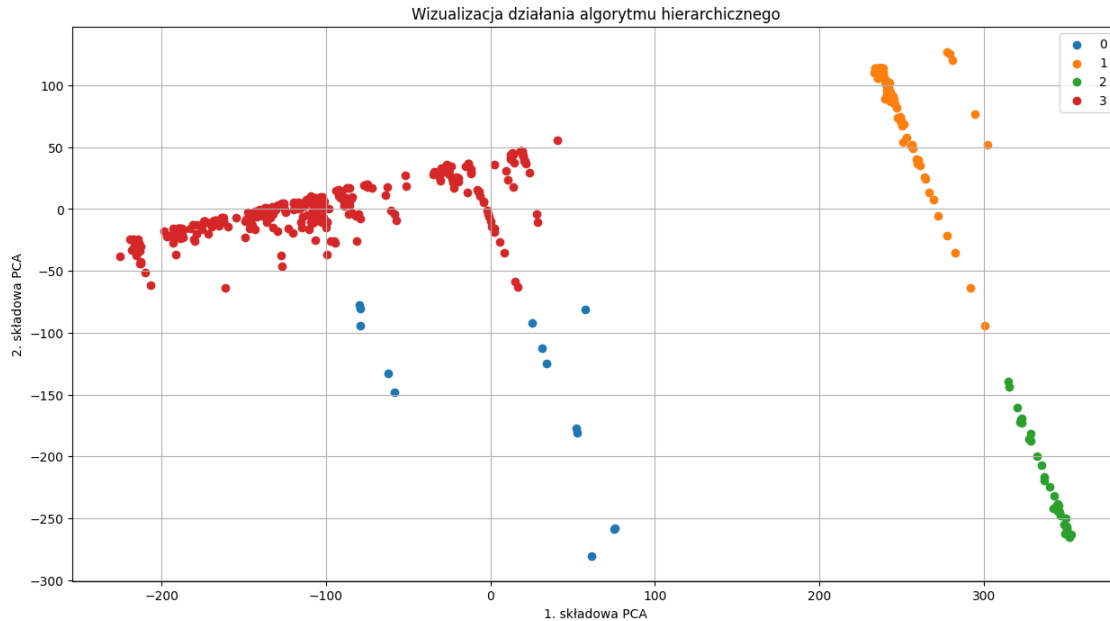
# Wykonanie algorytmu hierarchicznego:
clusterer = AgglomerativeClustering(n_clusters=n_clusters,
    ↪linkage=linkage_method)
clusterer.fit(data_norm)

# Zapisujemy sobie wynik przypisania danych do klastrow.
agclust_predictions = clusterer.labels_

# Wizualizacja za pomocą wcześniej wprowadzonej już funkcji
    ↪plot_decision_regions.
plt.figure(figsize=(15,8))
# Narysujemy wykres punktowy z rozdzieleniem danych na klasy.
for val in np.unique(agclust_predictions):
    # Generujemy maskę za pomocą której wybierzemy punkty należące do każdej z
    ↪klas.
    mask = agclust_predictions==val
    # Dla tak wszystkich punktów wybranych za pomocą odpowiadającej danej
    ↪klasie maski rysujemy wykres punktowy.
    plt.scatter(reduced_data[mask,0],reduced_data[mask,1], label=val)

# Oznaczenia osi i pozostałe elementy wykresu.
plt.xlabel('1. składowa PCA')
plt.ylabel('2. składowa PCA')
plt.title("Wizualizacja działania algorytmu hierarchicznego")
plt.grid()
plt.legend()
```

```
[12]: <matplotlib.legend.Legend at 0x1fb70f41d10>
```



Dla domyślnych danych uzyskaliśmy nieco inny podział za pomocą algorytmu hierarchicznego (względem algorytmu k średnich). Spróbujmy zatem znowu narysować, jak w zależności od klastra wygląda wykres pudełkowy metryki docelowej.

```
[13]: # W wizualizacji cech klastrów znowu będzie możliwy wybór wizualizowanej
      ↪ wielkości - zmienna docelowa to wartość 'target',
      # wymiar pochodzący ze zbioru danych natomiast, to odpowiednia nazwa wymiaru:
      #
      # - w zbiorze Boston Housing do dyspozycji są następujące wymiary:
      # 'CRIM' 'ZN' 'INDUS' 'CHAS' 'NOX' 'RM' 'AGE' 'DIS' 'RAD' 'TAX' 'PTRATIO' 'B'
      ↪ 'LSTAT'
      #
      # - w zbiorze Diabetes do dyspozycji są z kolei zmienne:
      # 'age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6'

      # Nazwa wizualizowanej wielkości:
      visualized_quantity_name = 'target'

      # Sprawdzamy, czy mamy wybrać zmienną target, czy kolumnę w tablicy data_norm.
      if visualized_quantity_name == 'target':
          visualized_quantity = target
      else:
          dim_index = np.where(dataset_dict['feature_names'] ==
          ↪ visualized_quantity_name)
          visualized_quantity = data_norm[:,dim_index].squeeze()
```

```

# Proces wizualizacji aczniemy od pustej listy.
clusterized_income_df = []

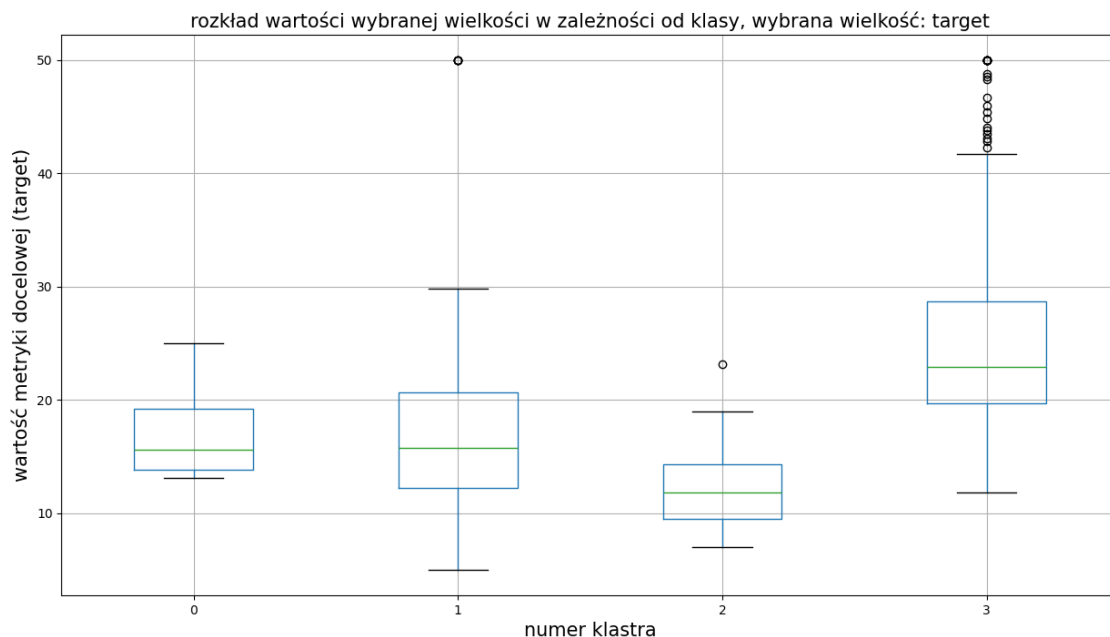
# W pętli dodajemy do listy wartości metryki docelowej (target), z której za-
    ↪pomocą maski
# wybieramy wartości powiązane z każdym z klastrów.
for val in np.unique(agclust_predictions):
    mask = agclust_predictions==val
    clusterized_income_df.append(visualized_quantity[mask])

# Na koniec konwertujemy listę do ramki danych (DataFrame)
clusterized_income_df = pd.DataFrame(clusterized_income_df).T

# Na podstawie tak przygotowanej ramki danych można w prosty sposób przygotować-
    ↪wykres pudełkowy za pomocą
# metody .boxplot().
plt.figure(figsize=(15,8))
clusterized_income_df.boxplot()
plt.xlabel('numer klastra', fontsize=15)
plt.ylabel('wartość metryki docelowej (target)', fontsize=15)
plt.title(f'rozkład wartości wybranej wielkości w zależności od klasy, wybrana-
    ↪wielkość: {visualized_quantity_name}',
          fontsize=15)

```

[13]: Text(0.5, 1.0, 'rozkład wartości wybranej wielkości w zależności od klasy, wybrana wielkość: target')



W przypadku algorytmu hierarchicznego otrzymane klasy można wręcz zinterpretować jako klasę nieruchomości najdroższych (skupisko o indeksie 0), klasę nieruchomości o średniej cenie (1) i klasę nieruchomości o najniższej wartości (indeks 2.).

2 Zadania do wykonania

Przed przystąpieniem do wykonywania zadań pamiętaj zapoznać się z kodem demonstrującym odczyt prawidłowej wartości k na podstawie dendrogramu.

2.1 Zadanie 1.

Zapoznaj się ze strukturą danych w zbiorze Boston Housing za pomocą wizualizacji na wykresie PCA (pierwsza wizualizacja od góry).

Zwróć uwagę, czy jesteś w stanie wizualnie wyróżnić odseparowane od siebie grupy punktów.

Zmieniając wartość zmiennej `coloring_variable_name` znajdź przykład 2 zmiennych (z wyjątkiem zmiennej `target`), które mogły mieć wpływ na to, że dane grupy punktów są od siebie odseparowane na wizualizacji. Podaj ich nazwy i wizualizacje w sprawozdaniu.

2.2 Zadanie 2.

Przejdź do sekcji przykładu zawierającej wykresy pudełkowe. Pozostawiając domyślny zbiór danych (Boston Housing) sprawdź za pomocą wykresów pudełkowych, jak kształtują się wartości poszczególnych kolumn zbioru danych w różnych grupach wygenerowanych przez algorytmy klasyfikacji nienadzorowanej. Czy jesteś w stanie na tej podstawie wywnioskować, jakie czynniki miały wpływ na takie a nie inne ceny mieszkań w poszczególnych klasach? Podaj 2 przykłady zmiennych (**z wyjątkiem zmiennej `target`**), dla których można zaobserwować taką sytuację. Podaj ich nazwy i wykresy pudełkowe w sprawozdaniu. Wykonaj tę czynność zarówno dla algorytmu `k` średnich, jak i hierarchicznego.

2.3 Zadanie 3.

Zbadaj zachowanie się algorytmu `k` średnich i hierarchicznego w zależności od sposobu normalizacji danych. Czy któryś ze sposobów normalizacji danych pozwolił na znalezienie klas, które jeszcze lepiej niż w zadaniu poprzednim wyróżniają się pod względem związanych z nimi cen mieszkań? (sprawdź to za pomocą wykresu pudełkowego) W trakcie badań może zmienić się optymalna liczba klas, dlatego w razie potrzeby należy ją zmienić za pomocą analizy wykresu intercji/dendrogramu.

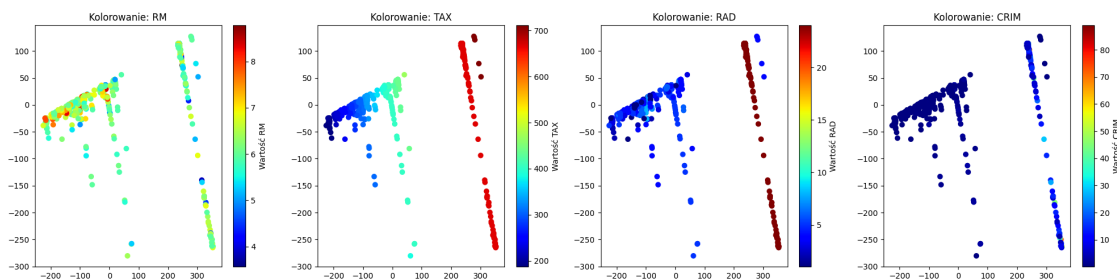


```
[7]: # Zadanie 1.  
# Wizualizacja wpływu poszczególnych zmiennych  
features_to_check = ['RM', 'TAX', 'RAD', 'CRIM']
```



```
plt.figure(figsize=(20, 5))
for i, feature in enumerate(features_to_check, start=1):
    plt.subplot(1, 4, i)
    dim_index = np.where(dataset_dict['feature_names'] == feature)
    feature_data = data_norm[:,dim_index].squeeze()

    scatter = plt.scatter(reduced_data[:,0], reduced_data[:,1], c=feature_data,
        cmap='jet')
    plt.colorbar(scatter, label=f'Wartość {feature}')
    plt.title(f'Kolorowanie: {feature}')
plt.tight_layout()
plt.show()
```



Wnioski - Zadanie 1: 1. Zdecydowanie widać odseparowane wyspy/chmury punktów na wykresie PCA. Główna bryła jest po prawej, ale wyraźnie odcięta od niej (rozciągnięta po lewej na dole) jest inna aglomeracja przypadków. Nienadzorowane metody szybko “chwycą” taki naturalny podział. 2. Zmienne, które napędzają ten podział, to chociażby wpisane wyżej **CRIM** (przestępczość) i **TAX** (podatki) – kolorowanie wyraźnie grupuje wartości duże w jednym rejonie, a małe w drugim. Podobnie ma się współczynnik **RAD** (dostępność autostrad), ewidentnie widać wyspę dużych i małych wartości odseparowanych od siebie. To one najpewniej decydują o geometrii w rzucie PCA.

```
[15]: # Zadanie 2.
# Wykresy pudełkowe dla dwóch zmiennych innych niż target - weźmy CRIM i LSTAT

def plot_boxplots(variable_name):
    # Pobranie wybranej kolumny
    dim_index = np.where(dataset_dict['feature_names'] == variable_name)[0][0]
    variable_data = data_norm[:, dim_index]

    # Dla hierarchicznego
    agclust_boxes = [variable_data[agclust_predictions == val] for val in np.
        unique(agclust_predictions)]
    # Dla k-means
    kmeans_boxes = [variable_data[k_means_predictions == val] for val in np.
        unique(k_means_predictions)]
```

```

plt.figure(figsize=(15, 4))

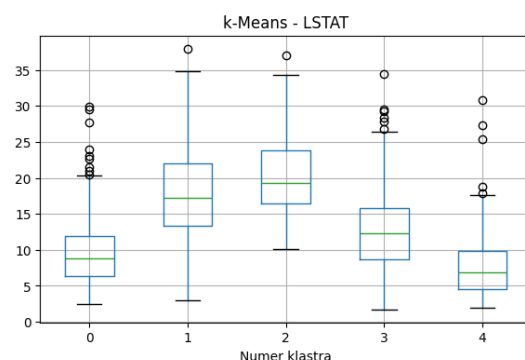
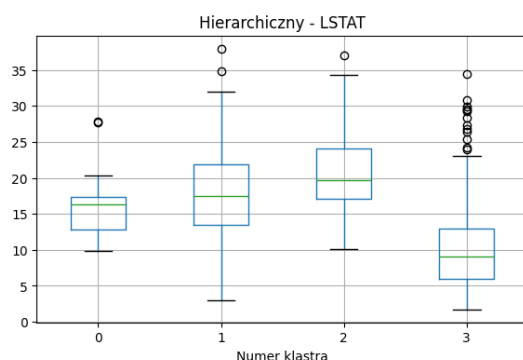
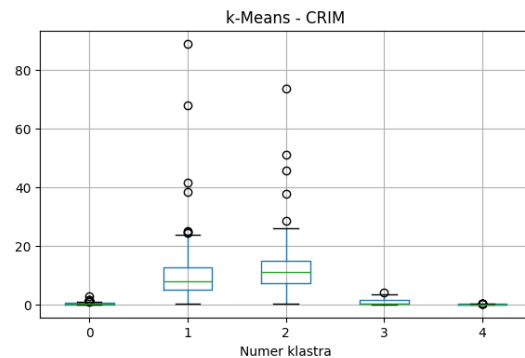
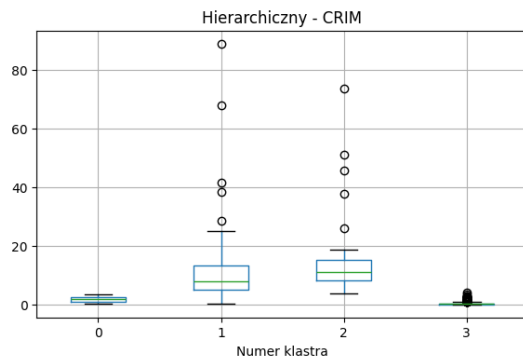
plt.subplot(1, 2, 1)
pd.DataFrame(agclust_boxes).T.boxplot()
plt.title(f'Hierarchiczny - {variable_name}')
plt.xlabel('Numer klastra')

plt.subplot(1, 2, 2)
pd.DataFrame(kmeans_boxes).T.boxplot()
plt.title(f'k-Means - {variable_name}')
plt.xlabel('Numer klastra')

plt.show()

plot_boxplots('CRIM')
plot_boxplots('LSTAT')

```



Wnioski - Zadanie 2: Algorytmy klasteryzacji całkiem nieźle zgrupowały mieszkania o podobnej charakterystyce, co widać np. w zmiennych: 1. **CRIM** (wskaźnik przestępczości): Widać na wykresach pudełkowych, że jeden z klastrów niemal w całości pochłania obszary o wysokim wskaźniku przestępczości, podczas gdy pozostałe utrzymują się blisko zera. To mocno zbija ceny

mieszkań w dół w tej konkretnej grupie. 2. **LSTAT** (% populacji z niższym statusem społecznym): Również tutaj mamy piękne “schodki” między klastrami. Klasy mieszkań tańszych i średnich charakteryzują się wyraźnie wyższym poziomem LSTAT, co potwierdza, że status okolicy wpływa na podział i ostatecznie cenę. Występuje to w obu badanych algorytmach.

```
[17]: # Zadanie 3.
# Wpływ normalizacji na podziały pod kątem docelowej wartości mieszkań
↳('target')

norm_data_unit = norm_unit_absval(data)
norm_data_std = norm_standardize(data)

def analyze_normalization(data_normalized, title_prefix):
    # Wyznaczamy k (k-means) za pomocą inercji - dla uproszczenia założymy k=4 z
    ↳wcześniejszych analiz
    # ale możemy to też szybko sprawdzić
    k = 4

    # KMeans
    km = KMeans(n_clusters=k, random_state=42)
    km_preds = km.fit_predict(data_normalized)

    # Agglomerative
    ag = AgglomerativeClustering(n_clusters=k)
    ag_preds = ag.fit_predict(data_normalized)

    km_boxes = [target[km_preds == val] for val in np.unique(km_preds)]
    ag_boxes = [target[ag_preds == val] for val in np.unique(ag_preds)]

    plt.figure(figsize=(15, 3))

    plt.subplot(1, 2, 1)
    pd.DataFrame(km_boxes).T.boxplot()
    plt.title(f'{title_prefix} | KMeans | k={k}')
    plt.ylabel('Cena mieszkania')

    plt.subplot(1, 2, 2)
    pd.DataFrame(ag_boxes).T.boxplot()
    plt.title(f'{title_prefix} | Agglomerative | k={k}')
    plt.ylabel('Cena mieszkania')

    plt.show()

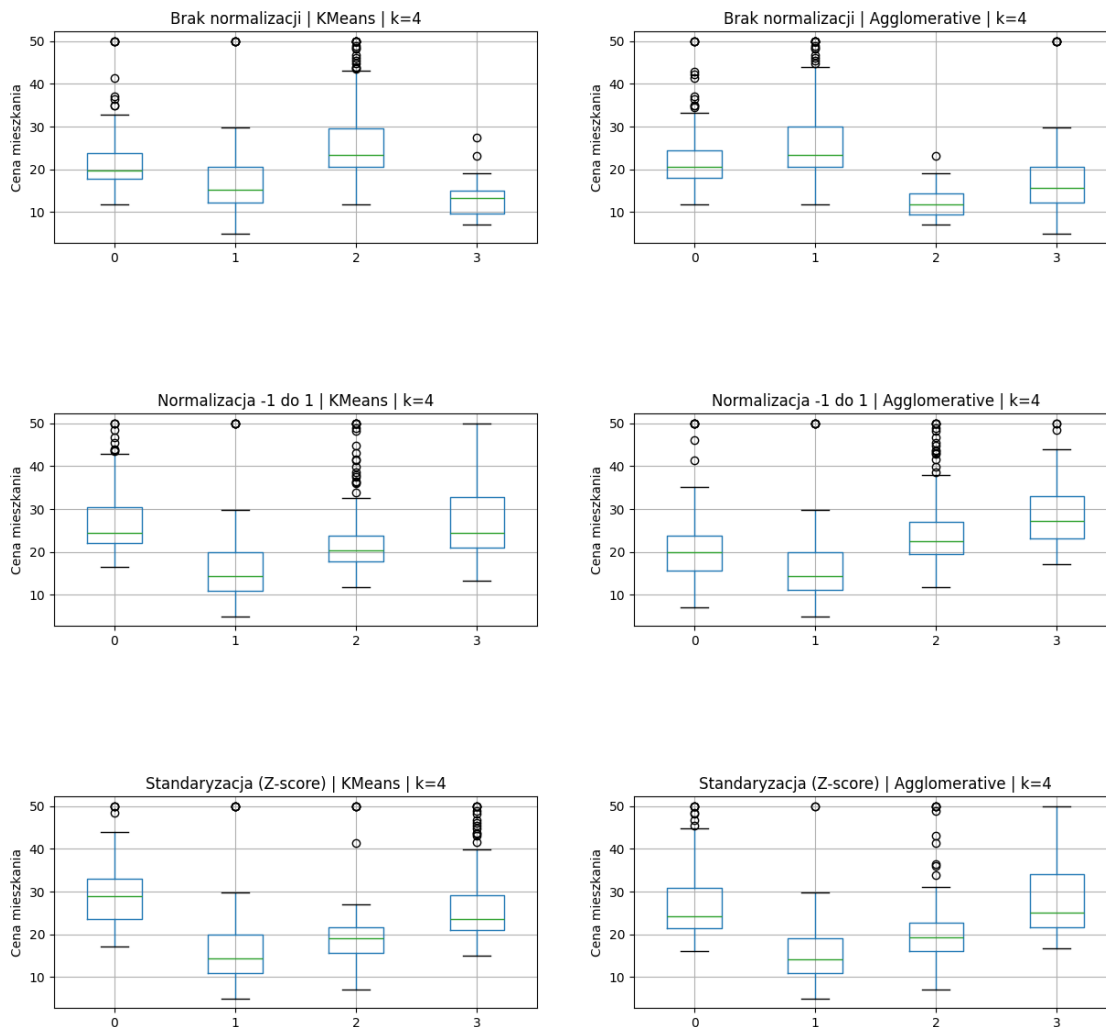
# Dla przypomnienia oryginalne dane (bez normalizacji):
analyze_normalization(data, 'Brak normalizacji')

# Różne normalizacje:
```

```

analyzer_normalization(norm_data_unit, 'Normalizacja -1 do 1')
analyzer_normalization(norm_data_std, 'Standaryzacja (Z-score)')

```



Wnioski - Zadanie 3: 1. Podejście do normalizacji ma gigantyczny wpływ na to, jak algorytmy odległościowe (k-Means, aglomeracyjny) radzą sobie z danymi. Bez normalizacji algorytmy dają się zdominować cechom o ogromnych rozstrzałach liczbowych (np. podatkowi TAX czy zmiennej B), olewając zupełnie proporcje innych cech. 2. Zastosowanie standaryzacji (Z-score) pozwala wyciągnąć ze zbioru prawdziwy sens. Na wykresach pudełkowych dla danych standaryzowanych widać bardzo wyraźne wręcz “schodki”, jeśli chodzi o mediany i przedziały cenowe dla poszczególnych klastrów ($k = 4$). Algorytmy pięknie odseparowały mieszkania bardzo tanie, tanie, średnie i te z klasy “premium”, podczas gdy bez normalizacji klastry cenowo potrafiły się mocno na siebie nakładać. Standaryzacja spisала się tu więc wzorowo.